AGI Expert Network

Distributed & Autonomous Intelligence Network

Whitepaper v1.0

Token: AGIEX

March 2026

agi.expert

This document describes the architecture, cryptoeconomics, and governance of the AGI Expert peer-to-peer research network and the AGIEX token.

Contents

1 Abstract

2 Introduction

3 Network Architecture

- 3.1 Node Anatomy
- 3.2 P2P Transport Layer
- 3.3 Capability System

4 Pulse Verification Protocol

- 4.1 Matrix Computation Challenge
- 4.2 Merkle Commitment Scheme
- 4.3 VRF Leader Election
- 4.4 Cross-Validation

5 AGIEX Token Economics

- 5.1 Token Parameters
- 5.2 Reward Formula
- 5.3 Reputation System
- 5.4 Emission Curve & Supply Projections
- 5.5 Staking Mechanics

6 On-Chain Program

- 6.1 Immutable Deployment
- 6.2 PDA Mint Authority
- 6.3 Instructions
- 6.4 Account Layout

7 Security Model

- 7.1 Ten On-Chain Security Layers
- 7.2 Slashing & Penalties

7.3 P2P Security

7.4 Sybil Resistance

8 Local Inference & Model Serving

8.1 Hardware Auto-Detection

8.2 Model Selection & GGUF Loading

8.3 OpenAI-Compatible API

9 Autonomous Research Pipeline

9.1 Research Projects

9.2 Mutation Strategies

9.3 CRDT Leaderboards

9.4 Agentic Coding

10 Governance

10.1 Council Structure

10.2 Proposal Lifecycle

10.3 Voting Mechanics

11 Conclusion

1 Abstract

We present AGI Expert, a fully decentralized peer-to-peer network where autonomous nodes conduct machine learning research, serve local inference, prove computational work, and earn AGIEX tokens through a trustless on-chain reward mechanism. The network requires no central coordinator, no cloud infrastructure, and no human approval for token minting. Each node auto-detects its hardware capabilities, loads quantized language models locally, participates in a 90-second pulse verification cycle, and contributes to eight parallel research projects through an evolutionary experiment pipeline. The Solana program governing AGIEX is deployed immutably with a Program-Derived Address (PDA) as sole mint authority — no private key exists that can mint tokens outside the verified proof pathway. Research direction is governed by a seven-seat multidisciplinary council and community-weighted voting, ensuring no single interest controls the network's trajectory.

2 Introduction

Contemporary AI research is concentrated in a small number of well-funded laboratories with exclusive access to compute, data, and talent. This concentration creates systemic risks: single points of failure, misaligned incentives, and barriers to participation for the global research community. Open-source model releases partially address the access problem but do not solve the coordination, incentive, or governance challenges.

AGI Expert proposes a different model: a flat, permissionless network of autonomous agents that collectively advance AI research. Every participant — from a laptop running a 1B parameter model to a server with an H100 — contributes compute, earns tokens proportional to their proven work, and has a voice in research governance. The system is designed around five core principles:

1. **No central authority.** Every node is architecturally identical. There are no coordinators, no privileged roles, no admin keys.
2. **Trustless economics.** The on-chain program is immutable. The mint authority is a PDA. Rewards are computed deterministically from verifiable proofs.
3. **Hardware-adaptive participation.** Capabilities are auto-detected and enabled at startup. Zero configuration required.

4. **Autonomous research.** Nodes generate hypotheses, run experiments, write papers, critique peers, and evolve strategies without human intervention.
5. **Diversified governance.** A seven-seat council spanning AI/ML, systems engineering, ethics, law, domain science, community advocacy, and security auditing governs research direction.

This whitepaper details the architecture, cryptoeconomics, and governance of the AGI Expert network and the AGIEX token.

3 Network Architecture

3.1 Node Anatomy

Every node in the AGI Expert network is a complete autonomous agent composed of eight subsystems:

SUBSYSTEM	RESPONSIBILITY
Identity	Ed25519 keypair generation and peer ID derivation
P2P + GossipSub	libp2p transport, topic-based pub/sub, mDNS and DHT discovery
Inference API	GGUF model loading via node-llama-cpp, OpenAI-compatible HTTP server
Research Pipeline	Hypothesis generation, experiment execution, paper writing, peer critique
Pulse Verification	VRF leader election, matmul challenge, Merkle commit-reveal, on-chain proof
CRDT Leaderboard	Yjs conflict-free replicated data structures for experiment results
Agent Brain	Autonomous 10-second decision loop coordinating all subsystems
Security Layer	Rate limiting, challenge-response, anomaly detection, peer scoring

All nodes are architecturally identical. There are no coordinator nodes, no master-worker relationships, and no privileged roles. The death of any node has zero impact on network operation.

3.2 P2P Transport Layer

The network uses libp2p with the following transport stack:

PROTOCOL	ENDPOINT	PURPOSE
TCP	`/ip4/0.0.0.0/tcp/4001`	Primary transport
WebSocket	`/ip4/0.0.0.0/tcp/4002/ws`	Browser node support
Circuit Relay	Dynamic	NAT traversal for restricted nodes

Discovery mechanisms: mDNS for local network peers, bootstrap peer list for initial connectivity, and Kademlia DHT for global peer discovery.

GossipSub topics carry all network communication:

TOPIC	CONTENT
`agi/pulse`	Pulse verification rounds and proofs
`agi/research/rounds`	ML training experiment results
`agi/coding/experiments`	Agentic coding experiment results
`agi/search/experiments`	Search engine experiment results
`agi/finance/experiments`	Financial analysis results
`agi/cause/skills`	Skills and tools discoveries
`agi/cause/inspiration`	Cross-project discoveries for mutation seeding
`agi/leaderboard/sync`	CRDT leaderboard synchronization
`agi/peers/announce`	Peer presence announcements
`agi/governance`	Proposals, votes, council updates

All GossipSub messages are signed with the node's Ed25519 key. Message format: `{data, peerId, nonce, timestamp, signature}`. Messages with timestamps outside a 5-minute window are rejected.

3.3 Capability System

At startup, the node scans its hardware and enables capabilities based on the following thresholds:

CAPABILITY	AUTO-ENABLED WHEN	REWARD WEIGHT
Inference	GPU detected, VRAM >= 4 GB	+10%
Research	GPU VRAM >= 6 GB, or CPU >= 4 cores + RAM >= 8 GB	+12%
Proxy	Public IP address detected	+8%
Storage	Free disk >= 50 GB	+6%
Embedding	CPU >= 2 cores, RAM >= 4 GB	+5%
Memory	RAM >= 8 GB, free disk >= 20 GB	+5%
Orchestration	CPU >= 4 cores, RAM >= 8 GB	+5%
Validation	CPU >= 2 cores	+4%
Relay	Public IP address detected	+3%

The maximum capability bonus for a node running all nine capabilities is 58%. Capability weights are stored as basis points on-chain (e.g., inference = 1000 bps).

4 Pulse Verification Protocol

The pulse protocol is the heartbeat of the network. Every ~90 seconds, nodes prove they are performing genuine computation and submit verifiable proofs to the Solana program.

4.1 Matrix Computation Challenge

Each pulse round requires the node to compute a matrix multiplication challenge. Let s denote the round seed (a 32-byte value derived from the previous round's hash):

```
Given:  s = SHA-256(previous_round_hash || round_number)

Generate:  A, B in R^{256x256}  where  A[i][j] = PRNG(s, i, j)

Compute:   C = A * B

           C[i][j] = sum_{k=0}^{255} A[i][k] * B[k][j]    for i, j in [0, 255]
```

The matrices A and B are generated deterministically from the seed using SHA-256 as a PRNG, ensuring any validator can independently reproduce the computation. The matrix dimension is fixed at `MATRIX_SIZE = 256`.

4.2 Merkle Commitment Scheme

The result matrix C is committed via a binary Merkle tree:

```
Let row_i = C[i][0..255]   for i in [0, 255]

Leaf_i  = SHA-256(row_i)                    // 256 leaves

Node_k  = SHA-256(Node_{2k} || Node_{2k+1})  // 8 levels

Root    = Node_0                             // 32-byte commitment
```

The node broadcasts its Merkle root as a commitment. During verification, 4 rows are challenged:

```
challenge_indices = { SHA-256(s || j) mod 256  :  j in [0, 3] }

For each challenged row i:
  - Node reveals:  row_i  and  merkle_proof_i  (8 sibling hashes)
  - Verifier:      recomputes row_i from A, B
                   verifies SHA-256(row_i) against merkle_proof_i
                   verifies proof_i resolves to committed root
```

The probability of cheating on all 4 challenges simultaneously is `(1/256)^4 = 1/4,294,967,296`, providing strong statistical guarantees without requiring full matrix verification.

4.3 VRF Leader Election

A Verifiable Random Function determines the round leader from the set of online peers:

```
peers = sort(connected_peer_ids)     // deterministic ordering

index = uint32(SHA-256(round_seed)) mod |peers|

leader = peers[index]
```

The leader coordinates the commit-reveal cycle for the round. All peers can independently verify the leader selection was correct given the same seed and peer set.

4.4 Cross-Validation

Before on-chain submission, proofs must be attested by independent validators:

PARAMETER	VALUE
Validators required per proof	3
Validation window	120 seconds
Minimum validator reputation	2,000 / 10,000
Honest validation reward	+3 reputation
Dishonest validation penalty	-200 reputation + 1 strike

Validators independently compute the matmul for the challenged rows and compare against the submitter's values. This cross-validation layer prevents collusion between a dishonest node and a complicit leader.

5 AGIEX Token Economics

5.1 Token Parameters

PARAMETER	VALUE
Token Name	AGI Expert
Ticker	AGIEX
Blockchain	Solana
Token Standard	SPL Token
Maximum Supply	1,000,000,000
Decimals	6
Smallest Unit	0.000001 AGIEX
Base Reward per Round	10 AGIEX
Pulse Interval	~90 seconds
Minimum Claim Interval	80 seconds
Mint Authority	PDA (no private key)
Upgrade Authority	None (deployed `--final`)
Pre-mine	None
Team Allocation	None
VC Allocation	None

Every AGIEX token in existence was minted by the on-chain program in response to a valid, cross-validated pulse proof. There is no pre-mine, no team allocation, no venture capital allocation, and no treasury controlled by any individual or entity.

5.2 Reward Formula

The reward for a single pulse round is computed as:

```
R = R_base  *  B_uptime  *  B_capability  *  M_reputation
```

Where each component is defined as follows:

5.2.1 Base Reward

```
R_base = 10 AGIEX
```

5.2.2 Uptime Bonus

The uptime bonus rewards long-running nodes with a logarithmic curve that provides diminishing returns:

```
B_uptime = 1 + 0.2 * ln(1 + h / 12)

where h = continuous uptime in hours
```

UPTIME	H	B_UPTIME
1 hour	1	1.016
12 hours	12	1.139
24 hours	24	1.220
7 days	168	1.546
30 days	720	1.826

The logarithmic shape ensures early uptime is rewarded substantially while preventing runaway bonuses for always-on infrastructure.

5.2.3 Capability Bonus

Nodes are rewarded for the services they provide to the network:

```
B_capability = 1 + sum(w_i)   for each enabled capability i

where w_i in {0.10, 0.12, 0.08, 0.06, 0.05, 0.05, 0.05, 0.04, 0.03}
```

With all 9 capabilities enabled: `B_capability = 1.58`. A minimal node with only validation enabled: `B_capability = 1.04`.

5.2.4 Reputation Multiplier

```
M_reputation = max(0.1, S_composite / 10000)

S_composite = 0.30 * S_honesty
            + 0.30 * S_quality
            + 0.20 * S_loyalty
            + 0.20 * S_consistency

Each sub-score is on a [0, 10000] scale.
```

The floor of 0.1 ensures that even low-reputation nodes receive 10% of the base reward, preventing complete exclusion while still strongly incentivizing good behavior.

5.2.5 Complete Reward Calculation

Combining all components, the reward per round for a node with reputation score *S*, uptime *h* hours, and enabled capability weights summing to *W*:

```
R = 10  *  (1 + 0.2 * ln(1 + h/12))  *  (1 + W)  *  max(0.1, S/10000)

Bounds:
  Minimum (new node, 1 cap, 0.1x rep):
    R_min = 10 * 1.0 * 1.04 * 0.1 = 1.04 AGIEX/round

  Maximum (30d uptime, 9 caps, perfect rep):
    R_max = 10 * 1.826 * 1.58 * 1.0 = 28.85 AGIEX/round
```

5.2.6 Daily Earnings Estimates

At 960 rounds per day (one round every 90 seconds):

NODE PROFILE	CAPABILITIES	REPUTATION	AGIEX/DAY
Browser, 2h/day	2-3	0.5x	~19
Browser, 24h	3-4	0.7x	~228
Desktop, 8 GB GPU	5-7	0.8x	~503
Server, 80 GB GPU	8-9	1.0x	~1,912

5.3 Reputation System

Each node maintains four on-chain reputation sub-scores, all on a [0, 10000] scale:

5.3.1 Honesty Score (weight: 30%)

```
S_honesty = (valid_proofs / (valid_proofs + invalid_proofs)) * 10000

Initial value: 1000
Per valid proof:   +5
Per invalid proof: -200
```

5.3.2 Quality Score (weight: 30%)

```
S_quality = base_quality - (consecutive_failures * 500)

base_quality = (recent_valid / (recent_valid + recent_invalid)) * 10000

QUALITY_WINDOW = 100   (last 100 proofs)
```

5.3.3 Loyalty Score (weight: 20%)

```
S_loyalty = min(10000,  (days_active / 30) * 10000)

Linear ramp: 0 at day 0, 10000 at day 30.
Threshold: 2,592,000 seconds = 30 days.
```

5.3.4 Consistency Score (weight: 20%)

```
S_consistency = max(0,  10000 - consecutive_misses * 200)

MAX_CONSECUTIVE_MISSES = 10 triggers additional reputation penalty.
Per missed round: -10 reputation.
```

5.3.5 Composite Score

```
S = 0.30 * S_honesty + 0.30 * S_quality + 0.20 * S_loyalty + 0.20 * S_consistency

Operating thresholds:
  S >= 500     required to earn rewards
  S >= 2000    required to validate peers
  S >= 2000    required to submit governance proposals
  S >= 500     required to vote on proposals
```

5.4 Emission Curve & Supply Projections

AGIEX has no halving schedule, no epoch-based emission changes, and no inflationary mechanics. The emission rate is purely a function of network participation: more nodes producing valid proofs means faster emission.

5.4.1 Emission Rate Formula

```
E_daily = N * rounds_per_day * R_avg

where:
  N              = number of active nodes
  rounds_per_day = 86400 / 90 = 960
  R_avg          = average reward per round across all nodes
```

For a network of *N* nodes with average reward *R_avg*:

```
E_daily(N) = N * 960 * R_avg
```

5.4.2 Time to Supply Cap

The total supply is hardcoded at `S_max = 1,000,000,000 AGIEX`. The time to reach the cap at constant network size:

```
T_cap = S_max / E_daily(N)

Assuming R_avg = 15 AGIEX (moderate rep, moderate caps):

  T_cap(10)    = 1,000,000,000 / (10  * 960 * 15)   = 6,944 days  = ~19.0 years
  T_cap(100)   = 1,000,000,000 / (100 * 960 * 15)   = 694 days    = ~1.9 years
  T_cap(1000)  = 1,000,000,000 / (1000 * 960 * 15)  = 69 days     = ~2.3 months
  T_cap(10000) = 1,000,000,000 / (10000 * 960 * 15) = 6.9 days    = ~1 week
```

5.4.3 Supply Function

Assuming a logistic network growth model `N(t) = N_max / (1 + e^(-k(t - t_0)))`, the cumulative supply at time *t*:

```
S(t) = min(S_max,  integral_0^t  E_daily(N(tau)) d(tau))

     = min(S_max,  960 * R_avg * integral_0^t N(tau) d(tau))

     = min(S_max,  960 * R_avg * N_max * [t + (1/k) * ln((1 + e^(-k*t_0)) / (1 + e^(-k*(t
```

This produces the characteristic S-curve: slow emission during early adoption, rapid emission during network growth, and hard termination at the 1B cap.

5.4.4 Projection Table

NETWORK SIZE	ROUNDS/DAY	DAILY EMISSION (EST.)	TIME TO 1B CAP
10 nodes	960	~144,000 AGIEX	~19 years
100 nodes	960	~1,440,000 AGIEX	~1.9 years
1,000 nodes	960	~14,400,000 AGIEX	~69 days
10,000 nodes	960	~144,000,000 AGIEX	~7 days

Estimates assume average reward of 15 AGIEX/round. Actual emission depends on each node's uptime, capabilities, and reputation.

5.4.5 Scarcity Dynamics

The fixed supply cap combined with participation-driven emission creates natural scarcity dynamics:

```
Share of total supply captured by node i:

  share_i = (sum of R_i over all rounds) / S_max

Early participants capture disproportionate share because:
  1. Fewer nodes competing for emission
  2. Reputation compounds over time (loyalty score)
  3. Uptime bonus grows logarithmically
  4. Supply cap creates hard ceiling
```

5.5 Staking Mechanics

After the 14-day maturation period, nodes must stake AGIEX as slashable collateral:

PARAMETER	VALUE
Minimum Stake	100 AGIEX (100,000,000 raw units)
Stake Lock Period	7 days after staking
Unstake Cooldown	7 days after request
Stake Vault	PDA-controlled (seed: "stake_vault")

Staking flow:

1. Node stakes >= 100 AGIEX. Tokens transferred to PDA-controlled vault. Locked for 7 days.
2. After 7 days, node can request unstake. Triggers 7-day cooldown.
3. After cooldown, node can withdraw tokens from vault.
4. During unstake cooldown, node continues earning but stake remains slashable.

6 On-Chain Program

6.1 Immutable Deployment

The Solana program governing AGIEX is deployed with the `--final` flag, which permanently removes the upgrade authority. Once deployed, the program cannot be modified, upgraded, or patched by any party. The code is the final authority.

6.2 PDA Mint Authority

The SPL token mint authority is a Program-Derived Address (PDA):

```
mint_authority = PDA(program_id, ["mint_authority"])

Properties:
  - No private key exists for this address
  - Only the program can sign with this authority
  - Only the submit_pulse_proof instruction invokes minting
  - The program code is immutable (--final)

Therefore:  tokens can only be minted through verified proofs.
```

6.3 Instructions

The program exposes 11 instructions:

#	INSTRUCTION	PURPOSE
1	`initialize`	Set supply cap, initialize program state
2	`register_node`	Create node account, begin maturation period
3	`stake`	Lock AGIEX as slashable collateral
4	`request_unstake`	Begin 7-day unstake cooldown
5	`withdraw_unstake`	Claim tokens after cooldown
6	`update_capabilities`	Update capability bitmask
7	`submit_pulse_proof`	Submit proof, pass 10 security layers, mint reward
8	`validate_peer`	Attest to a peer's proof correctness
9	`report_violation`	Report provable violations, trigger slashing
10	`heartbeat`	Prove liveness, track consecutive misses
11	`check_eligibility`	Read-only: query node's reward eligibility

6.4 Account Layout

Each registered node has an on-chain `NodeAccount` PDA (seed: `"node" || owner_pubkey`):

FIELD	TYPE	BYTES
owner	Pubkey	32
token_account	Pubkey	32
capabilities	u16 (bitmask)	2
registered_at	i64	8
last_claim_at	i64	8
last_claim_round	u64	8
last_heartbeat	i64	8
total_earned	u64	8
total_rounds_participated	u32	4
reputation	u32	4
honesty_score	u32	4
loyalty_score	u32	4
quality_score	u32	4
consistency_score	u32	4
is_matured	bool	1
is_banned	bool	1
maturation_proofs	u32	4
strikes	u8	1
consecutive_failures	u8	1
consecutive_misses	u8	1
cooldown_until	i64	8
last_failure_at	i64	8
stake_amount	u64	8
stake_locked_until	i64	8

FIELD	TYPE	BYTES
pending_unstake	u64	8
unstake_requested_at	i64	8
valid_proofs_submitted	u32	4
invalid_proofs_submitted	u32	4
validations_performed	u32	4
validations_received	u32	4
_reserved	[u8; 64]	64

Total account size: ~248 bytes + 8-byte Anchor discriminator = 256 bytes.

7 Security Model

7.1 Ten On-Chain Security Layers

Every `submit_pulse_proof` instruction passes through ten sequential security checks. Failure at any layer aborts the transaction:

#	LAYER	CHECK	CONSEQUENCE
1	Ban check	`is_banned == false`	Permanent rejection
2	Cooldown	`now >= cooldown_until`	Wait for exponential cooldown
3	Maturation	14 days + 100 valid proofs	No rewards (proofs tracked)
4	Stake bond	`stake >= 100 AGIEX`	Must stake post-maturation
5	Reputation gate	`reputation >= 500`	Must rebuild reputation
6	Round uniqueness	`round > last_claim_round`	RoundAlreadyClaimed error
7	Timing	`elapsed >= 80 seconds`	ClaimTooFrequent error
8	Merkle proof	Verify 4 challenged rows	5% stake slash + rep loss
9	Reputation scaling	Multiply reward by rep score	Reduced reward
10	Supply cap	`total_minted + reward <= 1B`	SupplyCapReached

7.2 Slashing & Penalties

7.2.1 Slashing Schedule

VIOLATION	STAKE SLASHED	REPUTATION	STRIKES
Invalid proof	5%	-200	+1
Collusion (fake validations)	50%	-200	+1
Sybil attack	100%	Zeroed	Permanent ban

5 strikes = permanent ban. Strikes are never cleared.

7.2.2 Exponential Cooldown

After each consecutive failure, the cooldown period doubles:

```
cooldown(n) = min(T_max,  T_base * 2^(n-1))

T_base = 300 seconds (5 minutes)
T_max  = 604,800 seconds (7 days)

  n=1:  5 min      n=4:  40 min     n=7:  5.3 hours
  n=2:  10 min     n=5:  80 min     n=8:  10.7 hours
  n=3:  20 min     n=6:  2.7 hours  n=11: 3.6 days (capped at 7d)
```

7.3 P2P Security

MECHANISM	PARAMETERS
Message signing	Ed25519 on all GossipSub messages
Timestamp freshness	Reject messages older than 5 minutes
Rate limiting	60 messages/min/peer, 50 proofs/hour/peer
Challenge-response	32-byte nonce every 300s, 10s timeout
Bot detection	Coefficient of variation < 5% over 20 samples = ban
Timing anomaly (fast)	< 50ms proof computation: -30 peer score
Timing anomaly (slow)	> 30s proof computation: -5 peer score
Auto-blocklist	Peer score < -100

7.4 Sybil Resistance

The system employs layered sybil resistance:

1. **Economic barrier:** 100 AGIEX minimum stake per node creates a linear cost for sybil attacks.

2. **Temporal barrier:** 14-day maturation + 100 valid proofs before earning. A sybil operator must maintain each fake node for two weeks.
3. **Computational barrier:** Each node must independently compute 256x256 matmul every 90 seconds. The cost of running N sybil nodes scales linearly with N.
4. **Detection:** 3+ nodes with identical Merkle roots in the same round are flagged. Penalty: -100 peer score per detected sybil node.
5. **Cross-validation:** Validators independently verify proofs. Collusion between sybil nodes is detected when validators disagree.

```
Cost of sybil attack with K nodes:

  C_sybil(K) = K * stake_min + K * compute_cost * T_maturation
             = K * 100 AGIEX + K * (electricity + hardware) * 14 days

Reward for K sybil nodes (assuming detection at rate d):

  R_sybil(K) = K * R_avg * (1 - d) - K * slash_rate * stake
             where d increases rapidly with K (detection is O(K^2) in pair checks)
```

For any non-trivial number of sybil nodes, the expected cost exceeds the expected reward, making sybil attacks economically irrational.

8 Local Inference & Model Serving

8.1 Hardware Auto-Detection

At startup, the node runs a hardware detection script that probes:

COMPONENT	DETECTION METHOD
GPU (NVIDIA)	`nvidia-smi --query-gpu=memory.total`
GPU (AMD)	`/sys/class/drm/*/device/mem_info_vram_total`
GPU (Apple)	`system_profiler SPDisplaysDataType`
CPU cores	`nproc`
RAM	`/proc/meminfo` or `sysctl hw.memsize`
Disk	`df -BM /`
Public IP	`curl -s ifconfig.me`

Results are written to `hw-profile.json` and used to auto-enable capabilities and select the recommended model.

8.2 Model Selection & GGUF Loading

The recommended model is selected based on available VRAM:

VRAM (GB)	RECOMMENDED MODEL	PARAMETERS
80+	qwen2.5-coder-32b	32B
48+	gemma-3-27b	27B
24+	gpt-oss-20b	20B
16+	gemma-3-12b	12B
12+	glm-4-9b	9B
8+	gemma-3-4b	4B
6+	gemma-3-4b	4B
4+	gemma-3-1b	1B
<4	None (CPU-only node)	-

Models are loaded from `./models/{model_name}.gguf` using `node-llama-cpp` with a context size of 4096 tokens.

8.3 OpenAI-Compatible API

Each node serves an HTTP API on port 8080:

ENDPOINT	METHOD	DESCRIPTION
`/v1/models`	GET	List loaded models
`/v1/chat/completions`	POST	Chat inference (streaming + non-streaming)
`/v1/embeddings`	POST	Text embeddings (384-dim, all-MiniLM-L6-v2)
`/health`	GET	Health check with model status
`/status`	GET	Full node status
`/leaderboard/{domain}`	GET	Top-10 experiment results for domain
`/governance`	GET	Governance status and active proposals

Streaming responses use Server-Sent Events (SSE). The API supports `max_tokens`, `temperature`, and full message history. Response format follows the OpenAI ChatCompletion specification with usage tracking.

9 Autonomous Research Pipeline

9.1 Research Projects

The Agent Brain runs a 10-second decision loop that cycles through eight parallel research projects:

PROJECT	PRIMARY METRIC	DIRECTION	DOMAIN
gpt2-tinystories	Validation loss	Lower is better	Language modeling
astrophysics	Validation loss	Lower is better	Scientific language modeling
search-engine	NDCG@10	Higher is better	Information retrieval
financial-analysis	Sharpe ratio	Higher is better	Quantitative finance
agentic-coding	Composite score	Higher is better	Code generation
skills-and-tools	Score	Higher is better	Tool use
academic-papers	Score	Higher is better	Scientific writing
p2p-network	Best result	Higher is better	Distributed systems

Each research cycle follows a five-stage pipeline: Hypothesis Generation, Experiment Execution, Paper Writing, Peer Critique, and Discovery Broadcast.

9.2 Mutation Strategies

Hypotheses are generated by applying mutations to the best-known configuration. The system implements 23 mutation strategies:

9.2.1 ML Architecture Mutations

STRATEGY	RANGE
learning_rate	0.5x to 3.5x of current value
context_length	{64, 128, 256, 512}
batch_size	{4, 8, 16, 32, 64}
model_width (d_model)	{64, 128, 256, 384, 512, 768, 1024}
depth (n_layers)	{2, 4, 6, 8, 12, 16}
normalization	{layernorm, rmsnorm}
position_encoding	{learned, rotary, sinusoidal}
activation	{gelu, silu, swiglu, relu}
weight_decay	{0.0, 0.001, 0.01, 0.05, 0.1}
init_scheme	{default, kaiming, xavier}
gradient_clip	{0.5, 1.0, 2.0, 5.0}
tied_embeddings	toggle
extended_training	duration * (1 + random)

9.2.2 Agentic Coding Mutations

STRATEGY	RANGE
lora_rank	{16, 32, 64, 128, 256}
lora_alpha	{rank, rank*2, rank*4}
lora_dropout	{0.0, 0.01, 0.05, 0.1}
lora_targets	3 preset module combinations
coding_data_mix	4 preset dataset combinations
finetuning_method	{lora, qlora, dora}
epochs	{1, 2, 3, 5, 8}
gradient_accumulation	{1, 2, 4, 8, 16}
max_seq_length	{2048, 4096, 8192}
generation_temperature	{0.0, 0.1, 0.2, 0.4, 0.6}

9.3 CRDT Leaderboards

Experiment results are tracked in five Yjs-based Conflict-Free Replicated Data Types (CRDTs), one per domain:

DOMAIN	METRIC FIELD	DIRECTION
research	`valLoss`	Ascending (lower wins)
search	`ndcg10`	Descending (higher wins)
finance	`sharpeRatio`	Descending (higher wins)
coding	`compositeScore`	Descending (higher wins)
causes	`score`	Descending (higher wins)

CRDTs converge automatically across all nodes without a central server. Each entry stores: peer ID, metric value, hypothesis, mutation strategy, configuration, GPU name, timestamp, and run number.

9.4 Agentic Coding

The agentic coding project fine-tunes Qwen 3.5 Coder 7B using LoRA adaptation:

9.4.1 Baseline Configuration

Base model	Qwen 3.5 Coder 7B (q4_k_m quantization)
Architecture	32 layers, 32 heads, d_model=4096, d_ff=11008, SiLU activation, RMSNorm, RoPE
LoRA rank	64
LoRA alpha	128
LoRA dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Learning rate	2e-4 with cosine schedule
Training	3 epochs, batch size 4, max 5000 steps, FP16

9.4.2 Composite Score Formula

```
S_composite = w_code * S_coding_avg + w_ml * S_ml_avg

where:
  w_code = 0.6     (coding benchmark weight)
  w_ml   = 0.4     (ML script benchmark weight)

  S_coding_avg = (S_humaneval + S_mbpp) / 2

  S_ml_avg = (S_mlbench + S_ds1000 + S_classeval) / 3

Baseline scores:
  S_humaneval  = 0.72    S_mlbench  = 0.45
  S_mbpp       = 0.68    S_ds1000   = 0.38
  S_classeval  = 0.52

  S_composite_baseline = 0.6 * (0.72 + 0.68)/2 + 0.4 * (0.45 + 0.38 + 0.52)/3
                       = 0.6 * 0.70 + 0.4 * 0.45
                       = 0.42 + 0.18
                       = 0.60
```

9.4.3 Mutation Impact Factors

```
LoRA ratio factor:     f_ratio = min(1, (alpha / rank) / 4) * 0.10
Data mix factor:       f_data  = min(1, |datasets| / 4) * 0.08
Method bonus:          f_method = { dora: +0.05, qlora: +0.02, lora: 0 }
Epoch factor:          f_epoch = min(0.06, ln(1 + epochs) * 0.02)

Mutated score = S_baseline * (1 + f_ratio + f_data + f_method + f_epoch + noise)
```

10 Governance

10.1 Council Structure

Research direction is governed by a seven-seat multidisciplinary council. Each seat represents a distinct domain of expertise, ensuring no single perspective dominates decision-making:

#	SEAT	EVALUATES
1	AI/ML Researcher	Technical feasibility, architecture validity, benchmark rigor
2	Systems Engineer	Infrastructure requirements, scalability, resource costs
3	Ethicist / Philosopher	Alignment, safety, societal impact, dual-use potential
4	Legal Counsel	Regulatory compliance, data licensing, intellectual property
5	Domain Scientist	Scientific rigor, methodology, reproducibility
6	Community Advocate	User impact, accessibility, practical value
7	Security Auditor	Attack surfaces, game theory, adversarial scenarios

Council members serve 90-day terms (7,776,000 seconds). Council approval requires 4 of 7 members.

10.2 Proposal Lifecycle

```
DRAFT  -->  REVIEW (3 days)  -->  VOTING (7 days)  -->  APPROVED / REJECTED  -->  ACTIVE
```

Proposal requirements: Proposer must have reputation >= 2,000. Proposals must include: objectives, benchmarks, data requirements, ethics statement, and success criteria.

10.3 Voting Mechanics

```
Vote weight = sqrt(reputation)

Rationale: Square root scaling reduces plutocratic influence.
  Node with rep 10000: weight = 100
  Node with rep  2500: weight =  50
  Node with rep   500: weight ~  22

Approval threshold: 66% of weighted votes
Quorum:             10% of active nodes must participate
Voting period:      7 days (604,800,000 ms)
Min rep to vote:    500 / 10000
```

This design ensures that high-reputation nodes have influence proportional to the square root of their reputation, rather than linearly. A node with 4x the reputation of another gets only 2x the voting power.

11 Conclusion

AGI Expert represents a fundamentally different approach to AI research infrastructure. Rather than centralizing compute, data, and governance in a single institution, the network distributes all three across a permissionless mesh of autonomous agents.

The design provides the following guarantees:

1. **Trustless token economics.** The AGIEX token has no pre-mine, no team allocation, and no admin keys. Every token is minted on-chain in response to a verified proof. The mint authority is a PDA, and the program is deployed immutably.
2. **Verifiable computation.** The pulse protocol combines matrix computation challenges, Merkle commitment schemes, VRF leader election, and multi-party cross-validation to ensure that only genuine compute earns rewards.
3. **Layered security.** Ten on-chain security layers, exponential cooldowns, slashing, and permanent bans make attacks economically irrational. P2P security adds rate limiting, challenge-response liveness, anomaly detection, and automatic blocklisting.
4. **Adaptive participation.** Hardware auto-detection and the nine-capability system allow any device from a browser to an H100 server to participate meaningfully and earn rewards proportional to its contribution.
5. **Autonomous research.** Eight parallel research projects run continuously across every node, with evolutionary mutation strategies, CRDT-based result sharing, and global leaderboards that converge without a central server.
6. **Diversified governance.** A seven-seat council spanning AI/ML, engineering, ethics, law, science, community advocacy, and security ensures that research direction reflects broad expertise rather than narrow interests.

The network is live on testnet. Anyone can join with a single command:

```
git clone https://github.com/agi-expert/agi.expert && cd agi.expert && ./start.sh
```

AGI Expert Network — AGIEX Token — Whitepaper v1.0 — March 2026
https://agi.expert